

Kadane Algorithm - DP for subarray

: 연속된 부분 배열(subarray)의 최대 합/곱 $O(n)$

Max sum for subarray
아이디어.

$$\begin{array}{r} -5 \quad 4 \quad 3 \\ \hline \text{합} = 2 \end{array}$$

$$\begin{array}{r} -5 \quad 4 \quad 3 \\ \hline \text{합} = 7 \end{array}$$

✗ 굳이 -5 는 포함시켜야 할까? 아니!

=> 현재까지 x_i 를 만들때, 우리는 이전 최대합이 0이었는지
아니면 나 자신으로 다시 시작할지 결정한다.

$$\text{current_max} = \max(x_i, \text{previous_max} + x_i)$$

$$\Rightarrow dp[i] = \max(\text{nums}[i], dp[i-1] + \text{nums}[i])$$

$$\Rightarrow \text{curr_max} = \max(\text{nums}[i], \text{curr_max} + \text{nums}[i])$$

$$\text{max_sum} = \text{nums}[0]$$

$$\text{curr_max} = \text{nums}[0]$$

for i in range(1, len(n)):

$$\text{curr_max} = \max(\text{nums}[i], \text{curr_max} + \text{nums}[i])$$

$$\text{max_sum} = \max(\text{max_sum}, \text{curr_max})$$

return max_sum

Max Product for Subarray

현재 합치는 다르면 $-x * y * -z$ 처럼 마지막 숫자가 이전의 최솟값을 최댓값으로 바꾸는 경우가 생김.

따라서, $curr_max$ or $curr_min$ 이 갱신되어야 함.

$$curr_max = nums[0]$$

$$curr_min = nums[0]$$

$$max_prod = nums[0]$$

for i in range(1, len(nums)):

$$temp = curr_max * nums[i]$$

↳ $curr_max$ 가 양이면 그냥 그대로 곱함.

$$curr_max = \max(\underbrace{nums[i]}_{\text{① 현재}}, \underbrace{curr_max * nums[i]}_{\text{② 이전 max * 현재}}, \underbrace{curr_min * nums[i]}_{\text{③ 이전 min * 현재}})$$

$$curr_min = \min(\underbrace{nums[i]}_{\text{① 현재}}, \underbrace{curr_max * nums[i]}_{\text{② 이전 max * 현재}}, \underbrace{curr_min * nums[i]}_{\text{③ 이전 min * 현재}})$$

$$res = \max(res, curr_max)$$

return res